Evaluation of Classical Retrieval Models and Segmentation Strategies for Markdown Search

Swoyam Pokharel
6CS030 Big Data
University Of Wolverhampton
Kathmandu, Nepal
me@pswoyam.com.np

***Abstract*—This paper evaluates Seekdown, a zero-dependency search engine built in Rust specifically targeted towards markdown-based technical documentation. This experiment compares 20 retrieval systems produced by combining four segmentation strategies (`section`, `chunk80`, `chunk120`, `chunk200`) with five classical ranking models: `Boolean`, `TF-IDF`, `BM25`, `BM25+`, and `Pivoted Length Normalization`. The evaluation uses 11,095 Markdown files, 60 queries, pooled relevance judgments, TREC-style qrels, and graded retrieval metrics. The results show that segmentation matters more than small ranking-model differences. Fixed-size chunks consistently beat section-based indexing on effectiveness. The best system was `chunk120_bm25plus` with `nDCG-at-10 = 0.7820`, while `chunk200_bm25` reached nearly the same quality (0.7710) with much lower median latency (4.24 ms instead of 7.93 ms). Section-based systems were fast, but the quality drop was too large. Overall, classical lexical retrieval is still practical for local markdown search, provided that the retrieval unit is chosen carefully.**

***Index Terms*—information retrieval, markdown search, document segmentation, classical lexical retrieval**

I. Introduction

Markdown documentation search sits in an odd position. Recently, most retrieval discussions have shifted toward dense retrieval and large language model pipelines [1], [2]. However, consumer-facing devices often need search systems that can run locally, respond quickly, and avoid the hardware requirements of neural information retrieval systems. In this case, lexical retrieval is not outdated. It remains useful because it is cheap, deterministic, able to run on most consumer devices, and easier to debug than neural-based approaches [3], [4].

Seekdown is a search engine targeted towards Markdown-based technical documentation. It has been tested specifically in this setting. The main challenge is that Markdown corpora are structured but very uneven. Especially in the context of technical documentation, a repository can contain a short README alongside a long API reference, with headings, lists, tables, and code blocks mixed together. This makes the actual retrieval unit significantly more important. So, with Seekdown, the aim is to answer two core questions:

- Which classical ranking model holds up best under extreme length variation and technical vocabulary?
- What is the right indexing unit for Markdown: a structure-aware section, or a fixed-size chunk?

The first question concerns document length normalization techniques such as `TF-IDF`, `BM25`, `BM25+`, and `Pivoted Length Normalization` [5], [6], [7], [8]. The second question is essentially a passage retrieval problem: should the system respect markdown boundaries, or should it force documents into uniform windows and hope that the window size works well? [9], [10], [11], [12], [13].

The evaluation was done using the Cranfield/TREC workflow [14]. A corpus of **11,095** Markdown files, containing **7,934,587** words, was paired with **60** queries. The top results from all systems were pooled, annotated into graded qrels, and scored using the standard retrieval metrics such as `MAP`, `MRR`, `P-at-10`, `Recall-at-10`, and `nDCG-at-10` [15], [16]. Scalability was measured separately by benchmarking build time, median latency, `p95` latency, and throughput across five corpus sizes.

This project is organized around five research questions:

- Which lexical ranking model performs best on Markdown retrieval?
- Does a structure-aware `section` segmentation outperform fixed-size chunking?
- Among fixed chunk sizes, which performs the best?
- How do segmentation strategies affect indexing and performance?
- Which configuration offers the best quality to performance trade-off?

Based on the findings, we can conclude that chunking wins on effectiveness, and chunk size seems to directly correspond with latency. `chunk120_bm25plus` is the best pure-quality configuration, but `chunk200_bm25` is a better practical trade-off when speed and quality both matter.

A. Contributions

This project:

- Provides a comparison of five different classical lexical ranking models, on a fairly large Markdown corpus.
- Compares structure-aware section indexing against three fixed-size chunking strategies.
- Adds evidence to the current chunk-sizing debate by testing whether smaller, larger, or section-aware chunks act as the best retrieval unit for technical documentation. This directly connects to recent concerns around over-segmentation, semantic fragmentation, and the loss of surrounding context in chunked retrieval systems [11], [17], [18].
- It reports both retrieval quality and performance under a growing corpus size.
- It produces a reproducible TREC-style markdown retrieval benchmark built from pooled judgments, giving a compact workflow for evaluating local documentation search.

II. Related Work

Seekdown revolves around three areas, namely: classical lexical ranking, segmentation / passage retrieval, and evaluation methodology.

For classical IR, it starts with `Boolean` retrieval. Although precise, it is very naive, because a term either matches or it does not [19]. Salton's vector space model and `TF-IDF` made retrieval less rigid by introducing partial matching [5]. But, the problem is that `TF-IDF` still rewards repeated terms too linearly. In long technical documents, seeing the same term ten more times does not always mean the document is ten times more useful. Usefulness is rarely a linear scale.

As such, probabilistic ranking functions were built to fix this. `BM25` remains the standard because it captures both term-frequency saturation and document-length normalization [6]. Modern benchmarking continues to show that a tuned lexical retrieval can still be extremely competitive in cases where exact terminology matters, such as code and technical documentation [3], [4].

Within the `BM25` family, document length is the long-running argument. Standard `BM25` can penalize long documents too aggressively, which often can suppress genuinely relevant long texts. `BM25+` addresses this by lower-bounding the normalization term so that long relevant documents still get meaningful credit when they match query terms [7]. `Pivoted Length Normalization` takes a different angle by pivoting the length penalty around a corpus-specific reference length [8]. Thus, a markdown corpus, with a fairly large length variability, is a direct stress test for these techniques.

Segmentation is the other half of the problem. Long documents rarely contain relevant information uniformly; usually, it's one small portion that directly matches some query. Passage retrieval is not a new topic; Callan proved this decades ago: search systems work better when they score these local relevant passages instead of treating the entire document as equally relevant [9].

Although fixed-size chunking provides a predictable structure, it can often break sentences midway, which in turn breaks any coherent ideas that were being said midway. Recent studies argue that this sort of naive chunking can fragment semantics and hurt retrieval quality [11], [17], [18].

Structured document retrieval argues that if a corpus already encodes good boundaries, said boundaries *should* matter. Early work showed that structural cues can improve retrieval when paired with `BM25`-style scoring [10]. More recent systems like SEAL and Markdown-focused benchmarks like MDEval convey the same point: structure and element awareness do matter for long technical documents [12], [13]. As such, Seekdown's `section` mode is a simple test of this idea; it treats headings as retrieval units.

Finally, the evaluation design follows standard TREC practice. Pooling allows relevance assessment at reasonable cost by judging only a candidate set drawn from multiple systems [14], [16]. `nDCG` is particularly appropriate here because judgments are graded and documentation search is top-heavy: users rarely scroll far, so top-rank placement matters more than deep recall [15].

III. System Design

Seekdown is a Rust-based command-line retrieval system for Markdown corpora. The design goal was straightforward: keep the pipeline inspectable, reproducible, and local. There are zero external runtime dependencies. Everything runs through one binary.

At the top level, the project is split into modules for benchmarking, CLI handling, evaluation helpers, indexing, loading, output formatting, query execution, ranking, and tokenization.

The CLI exposes five main commands:

- `search`: run one query and print ranked results.
- `run`: generate `TREC` run files for a query set.
- `bench`: benchmark repeated query execution and report build time, latency, and throughput.
- `pool`: create pooled candidate sets from multiple runs.
- `qrels`: convert annotated pools into `TREC` qrels.

A. Corpus Processing

The corpus consists of Markdown technical documentation under `dataset/`. At evaluation time, it contains 11,095 files and 7,934,587 words. Those files belong to the following projects:

```
┌ ~/Projects/seekdown/dataset on main
└ ❯ tree -L 1
.
├── axum
├── bat
├── book
├── cargo
├── clap
├── docker-docs
├── fd
├── fishtest
├── github-docs
├── hugoDocs
├── hyprland-wiki
├── json
├── kubernetes-website
├── mdBook
├── niri
├── nomicon
├── nushell.github.io
├── rayon
├── reference
├── ripgrep
├── rust-by-example
├── rustc-dev-guide
├── rust-clippy
├── rustfmt
├── rustlings
├── serde
├── sway
├── tokio
├── vscode-docs
└── wezterm
31 directories, 0 files
```

Seekdown supports two loading modes:

- `section`: headings define retrieval unit boundaries.
- `chunk`: documents are split into fixed-size token windows.

Three chunk sizes were evaluated: 80, 120, and 200.

B. Indexing and Retrieval Models

Documents are tokenized, segmented into retrieval units, and scored using one of five ranking models:

- `Boolean`
- `TF-IDF`
- `BM25`
- `BM25+`
- `Pivoted Length Normalization`

`Boolean` is included just as a baseline. `TF-IDF` is the classic weighted vector-space approach [5]. `BM25` is the default probabilistic baseline [6]. `BM25+` and `Pivoted Length Normalization` test two different responses to document length bias [7], [8].

For the test, every ranking model is paired with every segmentation family, yielding 20 different models.

C. Query Processing and Output Format

The effectiveness query set contains 60 short, developer-style information needs stored in `tests/queries/queries.csv`. Representative examples are shown in Table I.

TABLE I
Examples from the 60-query set

ID	Query text
`q001`	axum routing handlers
`q002`	axum middleware state management
`q003`	bat syntax highlighting themes
`q004`	bat line numbers and paging
`q005`	book markdown chapters and summary
`q006`	book publishing and output formats
`q007`	cargo dependencies features and workspace configuration
`q008`	cargo build release and test commands
`q009`	clap derive subcommands and argument parsing

Each system produces TREC-formatted run files so that scoring can be performed using the standard `trec_eval` toolkit. This makes Seekdown align with the established evaluation methodology used in the Text REtrieval Conference (TREC).

IV. Methodology

The experimental design follows the Cranfield/TREC paradigm [14]. The system matrix is fixed, the corpus and query set are fixed, judgments are pooled, and scoring uses standard IR metrics.

A. Evaluation Corpus and Query Set

The corpus contains 11,095 files and 7,934,587 words. The query set contains 60 short, task-focused queries. It is worth noting that the corpus is "single domain" in the broad sense that it consists of only technical documentation. With that said, it contains varied projects and writing styles, which keeps the benchmark realistic while preserving enough terminology overlap for lexical retrieval to be meaningful.

B. System Matrix

TABLE II
Frozen 20-system evaluation matrix.

Seg.	System IDs
section	`section_boolean` `section_tfidf` `section_bm25` `section_bm25plus` `section_pivoted`
chunk80	`chunk80_boolean` `chunk80_tfidf` `chunk80_bm25` `chunk80_bm25plus` `chunk80_pivoted`
chunk120	`chunk120_boolean` `chunk120_tfidf` `chunk120_bm25` `chunk120_bm25plus` `chunk120_pivoted`
chunk200	`chunk200_boolean` `chunk200_tfidf` `chunk200_bm25` `chunk200_bm25plus` `chunk200_pivoted`

The matrix combines four segmentation families with five ranking models, totaling 20 systems.

C. Relevance Judgments and Qrels Construction

Relevance judgments were produced through pooling, following standard TREC practice [14], [16]. The top-10 results from all 20 systems were pooled into one candidate set. After de-duplication, this produced 4,263 judged candidate rows across 60 queries. Each pooled item was assigned a relevance label. The final label distribution was 1,202 items at relevance 0, 728 at relevance 1, 887 at relevance 2, and 1,446 at relevance 3.

Of course, pooling has the obvious limitation: anything not residing in the top-10 from the pooled systems is unjudged. This is acceptable here because the goal is evaluation within the 20-system matrix, not the absolute recall over the entire corpus.

The annotated pool was converted into TREC qrels using Seekdown's `qrels` command, and effectiveness metrics were computed through the standard run-file/qrels workflow.

D. Metrics

Effectiveness is reported using `MAP`, `MRR`, `P-at-10`, `Recall-at-10`, and `nDCG-at-10`. `nDCG-at-10` is treated as the primary metric because judgments are graded and the intended use case is top-heavy [15].

Scalability is reported using build time, mean latency, median latency, `p95` latency, total wall time, and `QPS`. Median latency is prioritized as it better represents typical query behaviour.

E. Scalability and Performance Experiments

Performance experiments were run across five deterministic corpus sizes:

- `subset_2500`,
- `subset_5000`,
- `subset_7500`,
- `subset_10000`,
- `full_corpus (11088)`.

For each system and corpus size, benchmarks recorded index build time and repeated query execution over the 60-query set. Each benchmark used 20 repetitions, yielding 1,200 query executions per system and size pair.

The main objective of this experiment was to measure how different segmentation strategies affect index size and query processing cost as the corpus scales.

V. Results and Discussions

The pattern is consistent across the results: chunking improves retrieval quality, while smaller chunks cost more at query time.

A. Retrieval Effectiveness

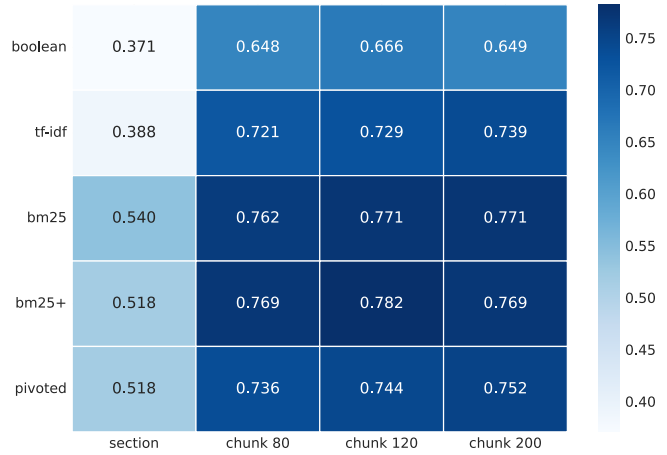

Fig. 1. nDCG-at-10 across all 20 systems

The best `nDCG-at-10` was achieved by `chunk120_bm25plus` at **0.7820**. It was followed closely by `chunk120_bm25` and `chunk200_bm25` (both 0.7710), then `chunk200_bm25plus` (0.7695) and `chunk80_bm25plus` (0.7692). The margins between the best probabilistic chunked systems exist, but are small. The larger gap is between chunked and section-based indexing.

TABLE III
Most effective systems and the strongest section baseline.

System	MAP	MRR	P10	nDCG	R10	Med.
`c120_bm25+`	0.1718	0.9500	0.8350	**0.7820**	0.1878	7.93
`c120_bm25`	0.1681	**0.9569**	0.8333	0.7710	0.1815	7.91
`c200_bm25`	0.1680	0.9519	0.8350	0.7710	0.1851	4.24
`c200_bm25+`	0.1656	0.9489	0.8283	0.7695	0.1826	4.02
`c80_bm25+`	**0.1738**	0.9431	**0.8400**	0.7692	**0.1911**	12.13
`sec_bm25`	0.1593	0.9583	0.7717	0.5405	0.1725	**3.96**

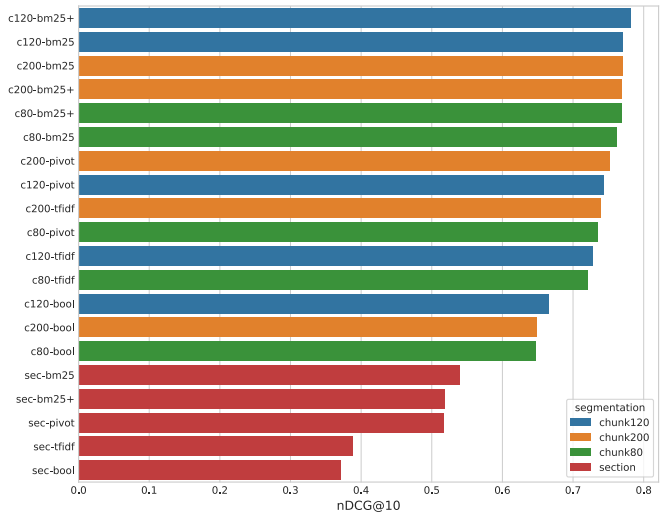

Fig. 2. Ranked full-system view of nDCG-at-10.

The average `nDCG-at-10` for `section_*` systems was 0.4671. `chunk80`, `chunk120`, and `chunk200` averaged 0.7270, 0.7383, and 0.7361. In this benchmark, authored sections were simply too broad to be good retrieval units.

Mean `nDCG-at-10` across all segmentations was 0.5832 for `Boolean`, 0.6441 for `TF-IDF`, 0.7111 for `BM25`, 0.7098 for `BM25+`, and 0.6875 for `Pivoted Length Normalization`. `Boolean` is too rigid, `TF-IDF` is a solid improvement, but the probabilistic methods lead.

Chunk size also matters. `chunk120` is the strongest family overall (0.7383 mean `nDCG-at-10`), with `chunk200` essentially tied (0.7361). `chunk80` falls behind slightly in quality, but much more in terms of latency.

One interesting finding is that different metrics have different winners. `chunk80_bm25plus` achieved the highest `MAP` (0.1738), the highest `P-at-10` (0.8400), and the highest `Recall-at-10` (0.1911), but it did not achieve the best `nDCG-at-10`. The summary is that `chunk80` surfaces many relevant items, but `chunk120` orders the highly relevant items better.

B. *Scalability and Query Performance*

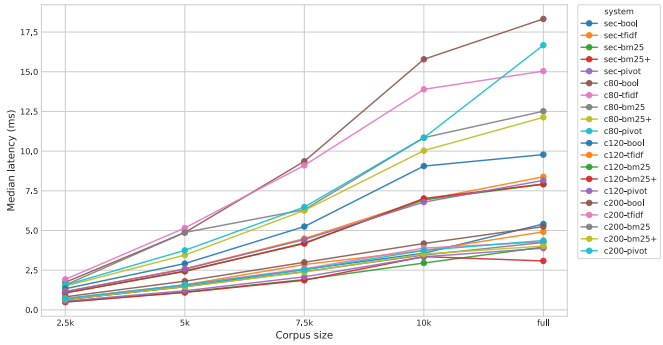

Fig. 3. Average query latency vs corpus size.

The results align with the expected intuition: finer segmentation produces more retrieval units, which increases query-time work.

On the full corpus, the average median latency of the `section_*` family was 4.25 ms. `chunk200_*` was similar at 4.43 ms. `chunk120_*` grew to 8.43 ms, and `chunk80_*` rose sharply to 14.94 ms.

Throughput follows the same ordering. The fastest full-corpus system was `section_bm25plus` at 314.14 `QPS`, while the slowest family overall was `chunk80_*`. Build time also shows a similar pattern. Full-corpus section systems built in roughly [1.06,1.11] seconds. Chunked systems clustered around [1.37,1.43] seconds, with `chunk80_pivoted` reaching 1.65 seconds. The key point is that chunk-size effects are much more visible at query time than at build time.

TABLE IV
FULL-CORPUS PERFORMANCE FOR THE STRONGEST SYSTEMS IN EACH SEGMENTATION FAMILY.

Seg.	System	Build ms	Median ms	P95 ms	QPS
section	`sec_bm25+`	1092.06	**3.08**	**5.31**	**314.14**
chunk80	`c80_bm25+`	1432.19	12.13	15.67	86.82
chunk120	`c120_bm25+`	1406.34	7.93	12.42	127.55
chunk200	`c200_bm25`	**1376.78**	4.24	6.79	237.39

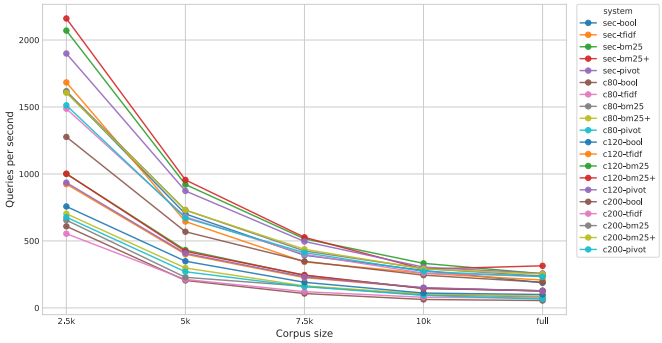

Fig. 4. QPS vs corpus size

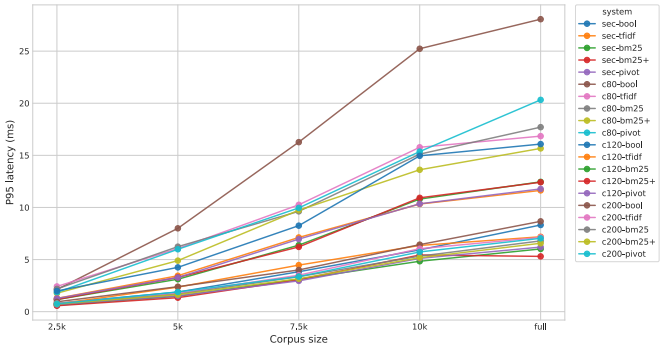

Fig. 5. P95 latency vs corpus size

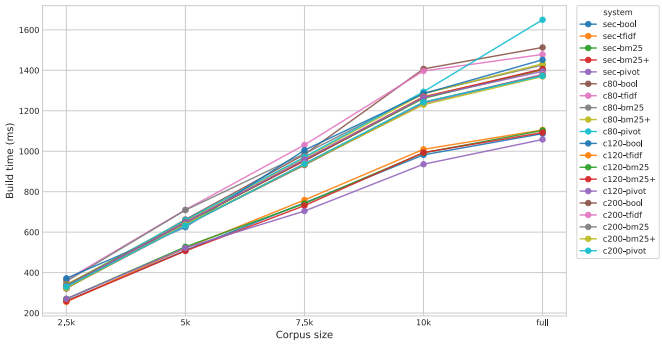

Fig. 6. Index build time vs corpus size

The growth curves present an interesting finding: not all chunking is equally expensive. `chunk200_*` matches section systems surprisingly closely, while `chunk80_*` separates quickly as the corpus grows.

C. Quality-Performance Trade-offs

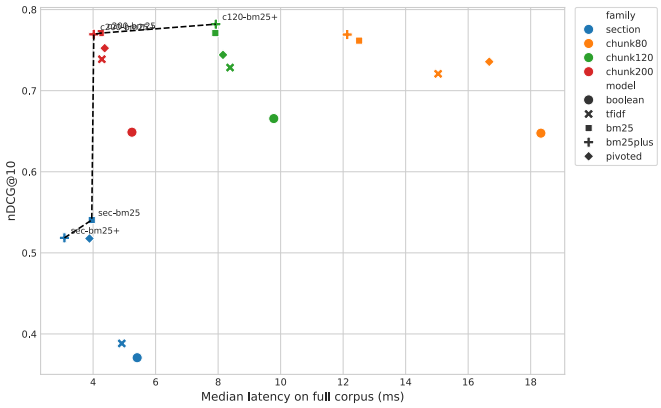

Fig. 7. Pareto frontier on the full corpus using nDCG-at-10 and median latency.

Figure 7 shows the trade-off as a Pareto frontier. The frontier contains five systems: `section_bm25plus`, `section_bm25`, `chunk200_bm25plus`, `chunk200_bm25`, and `chunk120_bm25plus`. The trade-off shows:

- If the only goal is top-rank effectiveness, `chunk120_bm25plus` is the clear winner.
- If speed dominates and quality can drop significantly, `section_bm25plus` is the fastest configuration.
- If both matter, `chunk200_bm25` is the most balanced point in the matrix.

`chunk200_bm25` reaches the same `nDCG-at-10` as `chunk120_bm25` and trails `chunk120_bm25plus` by only **0.0110** `nDCG-at-10`, while nearly halving median latency (**4.24** ms vs **7.93** ms) and boosting throughput (**237.39** `QPS` vs **127.55** `QPS`).

So the summary is pretty straightforward: `chunk120` squeezes out the bit of quality, but `chunk200` buys most of that quality at a much lower runtime cost, making it the practical trade-off.

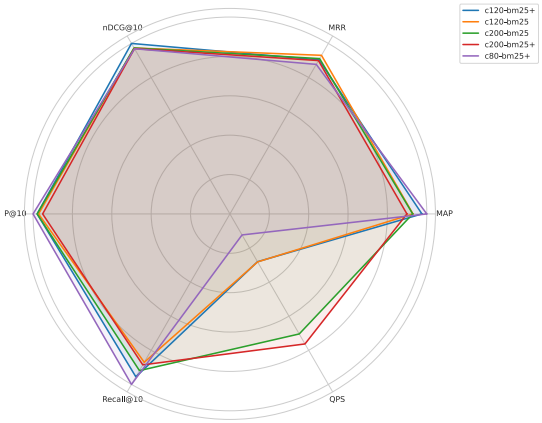

Fig. 8. Comparison of the top five systems across the metrics

D. Key Findings Summary

Thus, the five research questions can be answered directly:

- **RQ1:** Probabilistic ranking models were strongest. `Boolean` retrieval underperformed, `TF-IDF` improved on it, and `BM25` / `BM25+` dominated.
- **RQ2:** `section` segmentation did not beat chunking. It was consistently weaker on effectiveness.
- **RQ3:** `chunk120` was best on pure effectiveness, with `chunk200` coming extremely close.
- **RQ4:** Smaller chunks increased query latency and reduced throughput. `chunk200` scaled gently; `chunk80` did not.
- **RQ5:** `chunk120_bm25plus` is the best sheer quality system, but `chunk200_bm25` is the better practical trade-off in terms of quality to performance.

VI. Limitations

This study has some limitations. First, the corpus is domain-specific; it consists of only technical documentation, not general web retrieval. Second, the query set contains 60 queries, which is enough for a small controlled comparison but lacks in comparison to industrial benchmarks. Third, qrels were built through pooling, so documents outside the pool were not judged and are treated as non-relevant. Although this is standard TREC practice [14], it does mean that the recall measurements are bound only to the pooled candidate space. Fourth, the comparison is lexical only. No dense retriever, hybrid retriever, or neural reranker is included, so the results should be taken as a comparison among classical retrieval

techniques rather than a claim about beating modern neural systems.

VII. Conclusion

This paper evaluated 20 Markdown retrieval systems formed by combining four segmentation strategies with five classical ranking models. The results show that lexical retrieval still works well for local technical-document search, but the indexing unit does most of the real work.

The strongest system by effectiveness was `chunk120_bm25plus` (`nDCG-at-10` = **0.7820**). However, the main conclusion is about trade-offs rather than a single winner. Chunking consistently outperformed section-based indexing on effectiveness, and chunk size controlled most of the runtime.

From a practical standpoint, `chunk200_bm25` is the best default. It gives up very little effectiveness compared with the top system while still staying closer to section-level in terms of latency and throughput. Thus, we can conclude that better length normalization helps, but segmentation is where the large gains and the large costs actually come from.

References

[1] Z. Xu *et al.*, "A Survey of Model Architectures in Information Retrieval," *Transactions on Machine Learning Research (TMLR)*, 2025, [Online]. Available: https://arxiv.org/abs/2502.14822

[2] S. Chatterjee, "REGENT: Relevance-Guided Attention for Entity-Aware Multi-Vector Neural Re-Ranking," in *Proceedings of the 2025 Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region (SIGIR-AP 2025)*, Xi'an, China, 2025. doi: 10.1145/3767695.3769476.

[3] J. Killingback and H. Zamani, "Benchmarking Information Retrieval Models on Complex Retrieval Tasks," 2025, doi: 10.48550/arXiv.2509.07253.

[4] K. Kachhadiya and P. Patel, "Cross-Lingual Mathematical Information Retrieval with BM25 and LLM-based Pointwise Re-ranking," in *Forum for Information Retrieval Evaluation (FIRE 2025)*, Varanasi, India, 2025. [Online]. Available: https://ceur-ws.org/Vol-4173/T8-5.pdf

[5] G. Salton, A. Wong, and C.-S. Yang, "A vector space model for automatic indexing," *Communications of the ACM*, vol. 18, no. 11, pp. 613–620, 1975, doi: 10.1145/361219.361220.

[6] S. E. Robertson and S. Walker, "Okapi/Keenbow at TREC-8," in *Text REtrieval Conference (TREC-8)*, 1999. [Online]. Available: https://trec.nist.gov/pubs/trec8/papers/okapi.pdf

[7] Y. Lv and C. Zhai, "Lower-Bounding Term Frequency Normalization," in *Proceedings of the 20th ACM international conference on Information and knowledge management (CIKM '11)*, 2011. doi: 10.1145/2063576.2063584.

[8] A. Singhal, C. Buckley, and M. Mitra, "Pivoted document length normalization," in *Proceedings of the 19th annual international ACM SIGIR conference on Research and development in information retrieval*, 1996, pp. 21–29. doi: 10.1145/243199.243206.

[9] J. P. Callan, "Passage-level evidence in document retrieval," in *Proceedings of the 17th annual international ACM SIGIR conference on Research and development in information retrieval*, 1994, pp. 302–310. [Online]. Available: https://dl.acm.org/doi/10.5555/188490.188589

[10] Y. Champclaux, T. Dkaki, and J. Mothe, "ENHANCING HIGH PRECISION BY COMBINING OKAPI BM25 WITH STRUCTURAL SIMILARITY IN AN INFORMATION RETRIEVAL SYSTEM," in *Proceedings of the 11th International Conference on Enterprise Information Systems (ICEIS 2009)*, Milan, Italy, 2009. [Online]. Available: https://www.scitepress.org/papers/2009/20172/20172.pdf

[11] Y. Zhou, S. Wang, B. Koopman, and G. Zuccon, "Beyond Chunk-Then-Embed: A Comprehensive Taxonomy and Evaluation of Document Chunking Strategies for Information Retrieval," 2026.

[12] Z. Chen *et al.*, "MDEval: Evaluating and Enhancing Markdown Awareness in Large Language Models," in *Proceedings of the ACM Web Conference 2025 (WWW '25)*, Sydney, NSW, Australia, 2025. doi: 10.1145/3696410.3714674.

[13] X. Huang, Z. Ren, Y. Yu, Y. Zhou, Z. Chen, and Z. Wen, "SEAL: Structure and Element Aware Learning to Improve Long Structured Document Retrieval," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025. [Online]. Available: https://arxiv.org/abs/2508.20778

[14] E. M. Voorhees, "The philosophy of information retrieval evaluation," *Evaluation of cross-language information retrieval systems*. Springer, pp. 355–370, 2001. [Online]. Available: https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=151546

[15] K. Jarvelin and J. Kekalainen, "Cumulated gain-based evaluation of IR techniques," *ACM Transactions on Information Systems (TOIS)*, vol. 20, no. 4, pp. 422–446, 2002, doi: 10.1145/582415.582418.

[16] H.-T. Vu and P. Gallinari, "A Machine Learning based Approach to Evaluating Retrieval Systems," in *Proceedings of the Human Language Technology Conference of the NAACL, Main Conference*, New York City, USA: Association for Computational Linguistics, 2006, pp. 399–406. [Online]. Available: https://aclanthology.org/N06-1051.pdf

[17] A. V. Duarte, J. Marques, M. Graça, M. Freire, L. Li, and A. L. Oliveira, "LumberChunker: Long-Form Narrative Document Segmentation," 2024, doi: 10.48550/arXiv.2406.17526.

[18] Y. Liu, X. Xie, X. Wan, Y. Pan, and C. Wang, "Enhancing RAPTOR with semantic chunking and adaptive graph clustering," *Frontiers in Computer Science*, vol. 7, 2025, doi: 10.3389/fcomp.2025.1710121.

[19] Y. Gupta, A. Saini, and A. K. Saxena, "A SURVEY ON IMPORTANT ASPECTS OF INFORMATION RETRIEVAL," *Journal of Engineering and Technology (JET)*.